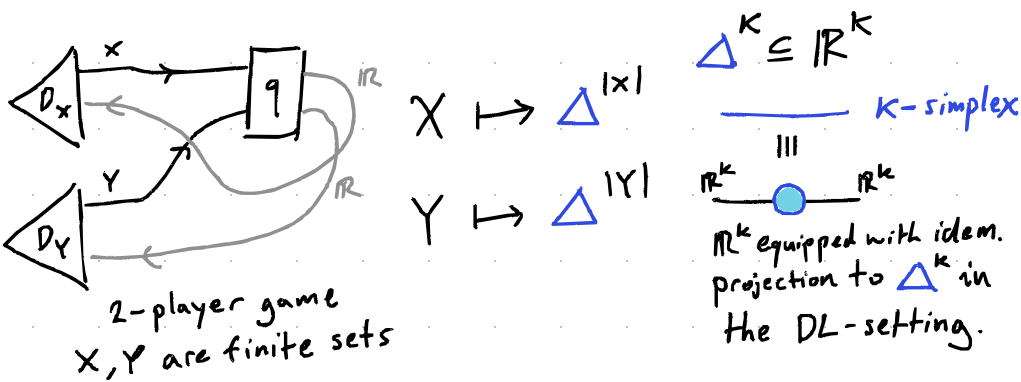


Deep learning for MSNE

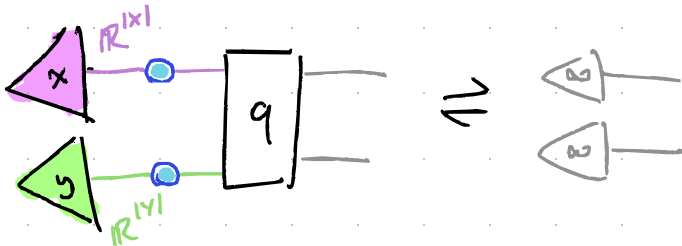
Idea:

- Deep learning is a kind of representation theory: you provide data and compute, and backpropagation will find satisfying representations of algebraic theories in smooth maps
- Solutions may not be compositional in open games, but the DL algorithms that compute those solutions are. (N.B. not cheating complexity here, because you have to pour a lot of compute into the DL process.)

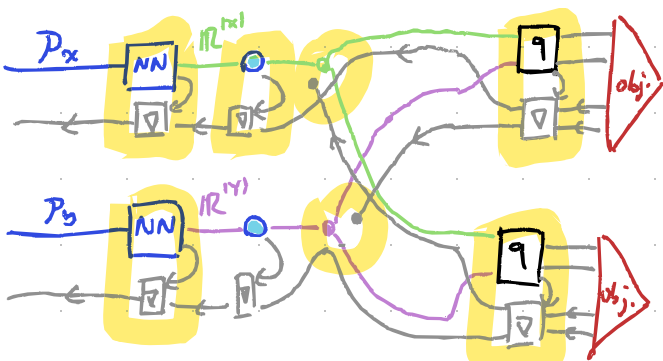


Starting with a 2-player game, we interpret mixed-strategies as points in the K -simplex, where K is the cardinality of the options X, Y . In DL-terms, we would interpret these as euclidean spaces of dimension K equipped with idempotent projections to the simplex (doable in two stages, first a linear projection down to the $x+y+z...=1$ hyperplane, followed by a second projection into the $[0,1]$ convex range for all variables.) These are depictable as just plain wires if we consider working with the Karoubi envelope of SmoothEuc, but I will depict the idempotents explicitly.

Now we just have to spell out what an MSNE is diagrammatically as a learning target. We want D_x and D_y to be learned vectors in their simplices that are "mutually locally optimal". i.e., let x and y be parameterised vectors (shaded pink and green), then the learning objective is for that x and y is just for both payoffs to be maximised (this can be recast later as minimising an objective function by isomorphism of $\mathbb{R}$ with $[0,1]$). As a first try, we have the following task:



This not quite enough, because x and y are here evolving in tandem, while MSNE requires each to consider other fixed. So let's spell out what happens during the learning process. Making parameters and the feedback of backpropagation explicit, the forward pass contains the currently parameterised positions (one for each of x and y), and the backward pass is gradient descent data (some jacobian). The trick here is that we have to "split" into two copies of q , where each copy treats one of the input MSs as fixed and the other as something to pass a gradient back to; this way, each MS is doing gradient ascent as if their opponent's MS is fixed.



Why this probably works and doesn't break any rules:

- Gradient descent here is just plotting paths through MS space, nudging a little towards better payoffs for both players, where each considers the other's MS (temporarily) fixed. So the updates terminate exactly at an MSNE.
- If this process does find a pair of mutually stable points, that's a numerical approx of an MSNE. Note, that's only finding *a* MSNE, not all of them. There's no way to guarantee with this process that you've found all of them.
- The search process is one-sided, i.e. it either succeeds or doesn't terminate. If it doesn't terminate, you can't know whether it's because you quit too early or if an MSNE just isn't there (unless there's a nonconstructive existence result for MSNEs). There could also be dynamic equilibria that end up being cycles in the "phase space" of two or more evolving MSs that you can get caught in.

Is this functorial? Probably! It seems pretty straightforward to take any description of an open game and turn it into a learning objective, where each decision process is a learner.